Egocentric Vision Hackathon — Problem Statements

Oct 30, 2026 · @Priyank

About the challenge

In 24 hours, your team will turn real first-person factory video into useful data about what workers' hands are doing. You will work out where the hands are, what they are holding, and which task step is happening.

The footage. The videos were recorded by cameras mounted on workers' heads in real factories. This is called *egocentric* video: you see exactly what the worker sees, and their hands are in view almost all the time.

Why it matters. Turning this kind of video into structured data has many real uses:

- Teaching robots to do manual tasks by watching people
- Checking that every assembly step was done, in the right order
- Building training material for new workers
- Spotting unsafe hand positions near machines

Today this data is mostly labelled by hand, which is slow and expensive. Your job is to automate it.

Why it is hard. The camera moves with the worker's head, so there is motion blur. Hands go out of view and get covered by tools and parts. Gloves change how hands look. Factory lighting is often harsh or uneven.

What matters most. Hand-tracking accuracy carries the biggest share of the judging, whichever problem you choose. You do not need a powerful computer to get it right: every problem can be solved on a normal laptop, and judges take your hardware into account.

The data

Every team gets the same set of real factory recordings from head-mounted cameras. The clips have **no labels**: finding the hands, objects and actions is your job.

What you receive

```
egohack-data/
  clips/        factory_001.mp4, factory_002.mp4, ...   # 20 clips, about 3
minutes each
  clips_480p/   the same clips, downscaled for laptops
  README.txt    resolution, frame rate, camera details if known
```

Some clips will be marked as **demo clips**. Every team must show results on these during the final presentation, so judges can compare teams on the same footage.

Data-use rules (read carefully)

This footage belongs to a real company and shows real workplaces. Every participant signs a data-use agreement at check-in.

- Use the data only for this hackathon.
- Do not post clips, frames or screenshots publicly. That includes GitHub, LinkedIn, YouTube and Instagram.
- Do not commit the video files to your Git repo. Keep them in a folder listed in `.gitignore`.
- You may send clips to cloud AI APIs (see Toolbox) to process them.
- Delete every copy of the data, including uploads to Colab, Kaggle and API file storage, when the event ends.
- If a clip shows a person's face, a screen, or company documents, do not include it in public posts about your project.

Breaking these rules means disqualification.

Problem statements

Choose **one** problem statement. You may combine several, but judges value one problem done well over many done badly. Each problem has a **minimum goal** that a beginner team can reach in the first few hours, and **bonus ideas** for strong teams.

ID	Title	Level	You produce
PS1	Hand tracking in factory video	Beginner	Hand boxes and finger points on every frame
PS2	What is the hand touching?	Intermediate	A timeline of when each hand grabs and releases objects
PS3	Task-step timeline	Intermediate	A list of actions with start and end times

ID	Title	Level	You produce
PS4	3D hand motion	Advanced	A 3D view of hand movement over time
PS5	Open innovation: video to insight	Open	A working tool or app built on any of the above

The suggested output formats below make your results easy to compare. They are recommended, not mandatory.

Hand-tracking accuracy is judged in every problem statement, because every other result builds on it. Even if you choose PS3 or PS5, show on the demo clips how reliably your system finds the hands.

PS1: Hand tracking in factory video

Why it matters. Almost every downstream use (robot learning, safety checks, step verification) starts with knowing exactly where the worker's hands are in every frame.

What to build. A program that takes a clip and, for every frame, finds each of the worker's hands, labels it left or right, and marks its 21 finger and wrist points. Each hand should keep the same ID from frame to frame.

Minimum goal. Run MediaPipe Hand Landmarker on a clip, save the results, and produce a video with the hands drawn on it.

Bonus ideas.

- Smooth out jittery points between frames
- Recover quickly when a hand leaves the view and comes back
- Fix frames where left and right get swapped
- Handle gloves, blur and dark areas better than the baseline, and show the before and after
- Report the share of frames where hands were found, and the speed in frames per second

Suggested output. `hands.jsonl`, one line per frame: `{"frame": 120, "t": 4.0, "hands": [{"side": "left", "bbox": [x1, y1, x2, y2], "score": 0.93, "kp2d": [[x, y], ...21 points]}]}`

How judges check accuracy. Judges watch your output on the demo clips frame by frame, paying most attention to the hard moments: motion blur, gloves, hands holding tools, overlapping hands, and hands leaving and re-entering the view. They check that

every hand is found, that the points sit on the right joints, and that left and right are never swapped. Measured numbers strengthen your case. For example, label hands on 100 frames yourself and report how many your system got right.

PS2: What is the hand touching?

Why it matters. Knowing when a worker picks up a tool or part, and puts it down, is the basis for checking work steps and teaching robots to grasp.

What to build. For each hand in each frame, decide whether it is touching nothing, a portable object (tool, part, box) or something fixed (machine, table, shelf). When it touches something, draw a box around the object, and name it if you can.

Minimum goal. Use hand tracking from PS1 plus an object detector. Call it a contact when two or more fingertips are inside or close to an object's box. Produce a grab/release timeline for one clip.

Bonus ideas.

- Follow the held object with a pixel mask (SAM 2)
- Name objects with an open-vocabulary detector or a vision-language API
- Count how often each tool is used per clip
- Flag hands close to a moving machine part as a safety warning

Suggested output. `contacts.csv`: `clip, t_start, t_end, hand, state, object_name`

PS3: Task-step timeline

Why it matters. A factory task is a sequence of steps. A timeline such as "0:12–0:20 pick up screwdriver, 0:20–0:41 fasten screws" lets a company check that steps happen in order and measure how long each takes.

What to build. A program that splits each clip into actions. Each action has a start time, an end time, a verb (pick, place, fasten, inspect…), an object, and the hand used.

Minimum goal. Send a clip to a video-understanding API (such as Gemini) and ask for a JSON timeline. Display it next to the video.

Bonus ideas.

- Use grab and release moments from PS2 to make the start and end times more precise
- Build a version that runs fully offline, using a small local model
- Compare two workers doing the same task: who is faster, and which steps differ?
- Spot steps that are missing or out of order

Suggested output. `actions.csv`: `clip, start_s, end_s, verb, object, hand`

PS4: 3D hand motion

Why it matters. Robots need hand motion in 3D, not just on a flat image, to copy a movement.

What to build. Estimate each hand's 3D shape and position over time, and show it as a 3D animation or plot next to the original video.

Minimum goal. Use MediaPipe's 3D "world" landmarks to draw a 3D skeleton of the hand for a short clip.

Bonus ideas.

- Recover a full 3D hand mesh with HaMeR or WiLoR (needs a free cloud GPU)
- Place the hands in a fixed "world" frame that stays still while the head moves (HaWoR or a SLAM tool)
- Export the hand trajectory in a format a robot simulator could load

There is no ground-truth 3D data, so judges look at how stable, smooth and believable the motion looks on the demo clips.

PS5: Open innovation, video to insight

Why it matters. Companies have hours of this footage and no easy way to use it.

What to build. Any working tool that turns raw factory video into something useful, using ideas from PS1–PS4.

Example ideas.

- Search clips in plain English, such as "show every time someone uses a drill with the left hand"
- A dashboard of time spent per task step, per clip
- An automatic quality filter that drops clips that are too blurry or have no hands
- A training-video generator that cuts and captions the key steps of a task
- A safety monitor that flags risky hand positions

What judges look for. A real user, a working demo on the demo clips, and a clear story about the value.

Choose your compute path

Pick the path that matches your hardware. You can mix them, for example by running MediaPipe on your laptop and a heavier model on a free cloud GPU. Tell the judges which you used; efficiency is scored against the hardware.

Organisers do not provide GPUs or API credits. Everything in this guide works with your own laptop and free tools.

	Laptop CPU path	Free cloud GPU path
Hardware	Your own laptop, 8 GB+ RAM, no graphics card needed	Google Colab (free tier) or Kaggle Notebooks (free weekly GPU hours)
Video to use	`clips_480p/`	`clips/` at full resolution
Frames to process	Every 2nd or 3rd frame; fill the gaps by interpolation	Every frame
Hand tracking (PS1)	MediaPipe Hand Landmarker; runs in real time	WiLoR or HaMeR for higher accuracy in blur and with gloves
Objects and contact (PS2)	Small YOLO model + MediaPipe hands + simple rules	Grounding DINO or OWLv2 for any object name, SAM 2 for masks
Task steps (PS3)	Video API call (works from any laptop)	Video API, or an open video-language model such as Qwen2.5-VL
3D hands (PS4)	MediaPipe 3D world landmarks	HaMeR or WiLoR mesh; HaWoR for world coordinates
Speed tricks	ONNX Runtime or OpenVINO; run heavy steps only when something changes	Process frames in batches; half precision (FP16)

Before you start on Colab or Kaggle:

- Free GPU sessions disconnect after some hours of use, and Kaggle limits GPU hours per week. Save results to Google Drive or Kaggle output often.
- Model weights can be several GB. Download them in the first hour, not the last.
- Check the latest free-tier limits on the Colab and Kaggle sites; they change.

Toolbox

There are three kinds of tools. Strong teams usually combine them: local models for **where** the hands are, and AI APIs for **what** is happening.

Fast models that run on a laptop

Tool	What it gives you	Good to know
MediaPipe Hand Landmarker	21 points per hand, left/right label, rough 3D points	The easiest start. Use its `VIDEO` mode for clips. Weaker with gloves, blur and hands holding objects.
YOLO (Ultralytics)	Boxes around objects	Small versions run fine on CPU; export to ONNX for speed
RTMPose / RTMDet (MMPose)	Fast body and hand points	An alternative to MediaPipe with small CPU-friendly versions
ByteTrack or OpenCV trackers	Keeps the same ID on a hand or object across frames	Lets you skip frames and fill the gaps

Strong models (use a free Colab or Kaggle GPU)

Model	What it gives you	Good to know
WiLoR	Finds hands and builds a 3D hand mesh	Faster than HaMeR, with its own hand detector
HaMeR	Accurate 3D hand mesh from a single image	Very robust; needs hand boxes as input
HaWoR	Hand motion in fixed world coordinates from head-camera video	Best fit for PS4 bonus; the most complex to set up
SAM 2	Cuts out and follows any object through a video	Click or draw a box once, and it tracks the object
Grounding DINO / OWLv2	Finds objects from a text name, like "wrench"	No training needed
Qwen2.5-VL, InternVL	Open video-language models that describe what happens	Offline alternative to an API for PS3

Even the strongest hand models still miss hands, swap left and right, or bend hidden fingers wrongly when hands overlap. Showing how you detect and fix these mistakes impresses judges.

AI APIs for understanding video

Send the whole video. The Gemini API accepts video files and can answer questions about them, including timestamps. Google AI Studio gives a free API key; check its current rate limits. By default Gemini looks at about **1 frame per second**, so it can miss fast hand movements. The docs explain how to raise this.

Send selected frames. Other AI models that accept images can be sent a few frames with their timestamps. This works well if you pick only the frames around a grab or release. Most other providers need paid credit.

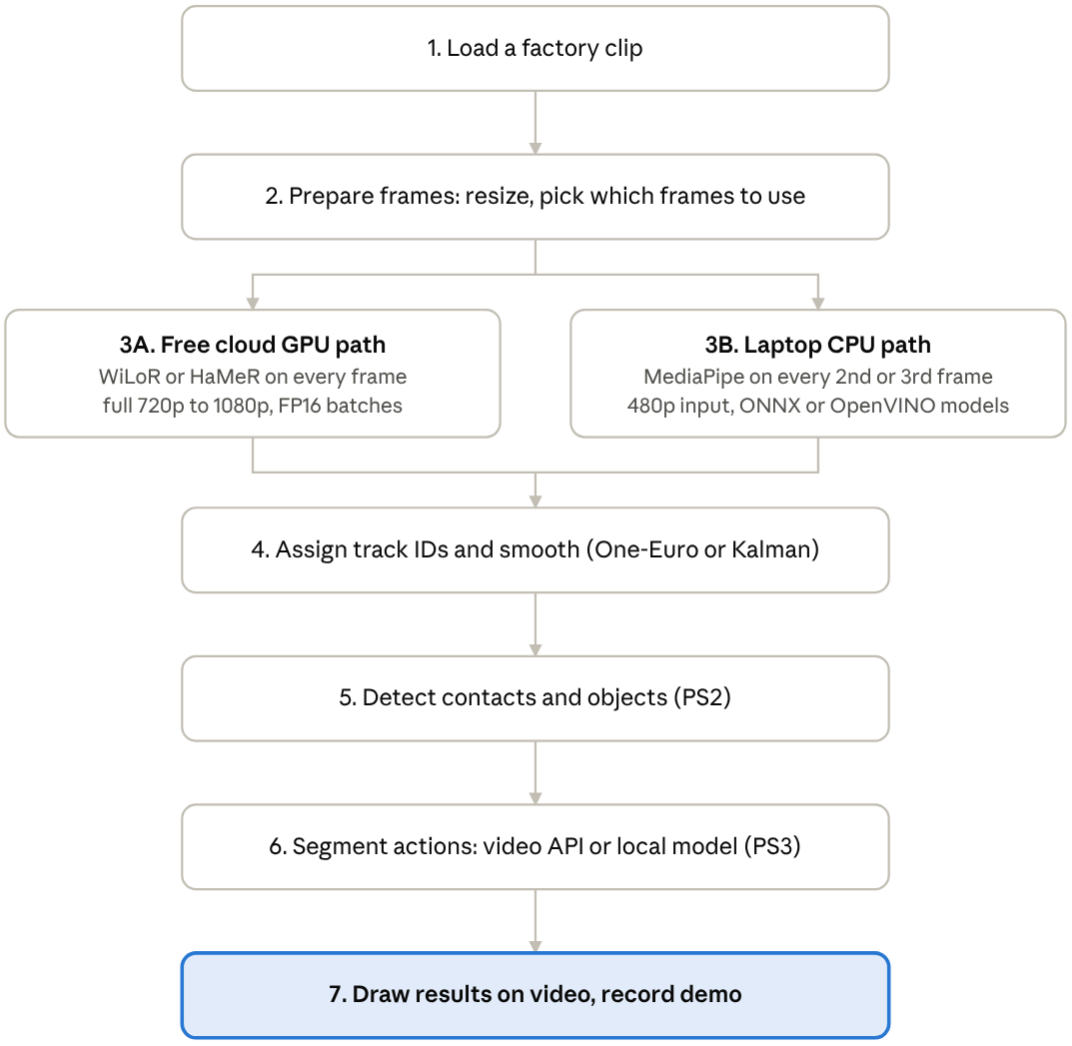
Use APIs carefully:

- Use the API for meaning (which action, which tool). It is not good at exact finger positions.
- Save every response to a file so you never pay or wait twice for the same clip.
- Expect rate limits on free tiers; process clips one at a time and retry slowly.
- Never put your API key in your code or repo. Use an environment variable.
- Delete uploaded files from the API's storage when the event ends (see Data-use rules).

Getting-started guide

Follow steps 0, 1–2, 3B and 7 to have a working demo within about 3 hours. Then spend the rest of the time on the bonus ideas for your problem.

Both tracks share one pipeline; only the hand model differs



starter pipeline · 7 steps, laptop and cloud GPU paths split at step 3

Only step 3 differs between the laptop and cloud GPU paths. PS1 teams stop after step 4, PS2 after step 5, and PS3 can jump from step 2 straight to step 6.

Step 0: Set up (do this first)

```
python -m venv egohack && source egohack/bin/activate     # Windows:
egohack\Scripts\activate
pip install mediapipe opencv-python numpy pandas tqdm google-genai
curl -L -o hand_landmarker.task \
  https://storage.googleapis.com/mediapipe-
models/hand_landmarker/hand_landmarker/float16/latest/hand_landmarker.task
```

For the cloud GPU path, open a Colab or Kaggle notebook with a GPU, then follow the setup in the WiLoR or HaMeR repo README.

Steps 1–2: Load a clip and pick frames

```
import cv2

def frames(path, step=1, width=None):
    """Yield (frame_index, time_in_seconds, image) for every `step`-th
frame."""
    cap = cv2.VideoCapture(path)
    fps = cap.get(cv2.CAP_PROP_FPS)
    i = 0
    while True:
        ok, img = cap.read()
        if not ok:
            break
        if i % step == 0:
            if width:
                img = cv2.resize(img, (width, int(img.shape[0] * width /
img.shape[1])))
            yield i, i / fps, img
        i += 1
```

On a laptop, use `clips_480p/` with `step=2` or `3`. On a GPU, use `clips/` with `step=1`.

Step 3B: Track hands with MediaPipe (laptop)

```
import json, mediapipe as mp
from mediapipe.tasks import python as mpp
from mediapipe.tasks.python import vision

opts = vision.HandLandmarkerOptions(
    base_options=mpp.BaseOptions(model_asset_path="hand_landmarker.task"),
    running_mode=vision.RunningMode.VIDEO,
    num_hands=2,
    min_hand_detection_confidence=0.4,
    min_tracking_confidence=0.4)

with vision.HandLandmarker.create_from_options(opts) as lm,
open("hands.jsonl", "w") as out:
    for i, t, img in frames("clips_480p/factory_001.mp4", step=2):
        h, w = img.shape[:2]
        mp_img = mp.Image(image_format=mp.ImageFormat.SRGB,
                          data=cv2.cvtColor(img, cv2.COLOR_BGR2RGB))
        res = lm.detect_for_video(mp_img, int(t * 1000))   # time in ms, must
increase
        hands = []
        for pts, hd, wpts in zip(res.hand_landmarks, res.handedness,
```

```
res.hand_world_landmarks):
            kp = [[p.x * w, p.y * h] for p in pts]
            xs, ys = zip(*kp)
            hands.append({"side": hd[0].category_name.lower(),   # check: may
need swapping
                          "score": hd[0].score,
                          "bbox": [min(xs), min(ys), max(xs), max(ys)],
                          "kp2d": kp,
                          "kp3d": [[p.x, p.y, p.z] for p in wpts]})  #
metres, around the hand
        out.write(json.dumps({"frame": i, "t": t, "hands": hands}) + "\n")
```

Two things to check. MediaPipe labels left and right as if the image were a mirrored selfie. Head-camera video is not mirrored, so watch your output and swap the labels if they come out backwards. Also, the 3D "world" points are measured around the hand's own centre, not from the camera.

Step 3A: Track hands with WiLoR or HaMeR (cloud GPU)

1. Clone the repo in Colab or Kaggle and run its demo on a few frames to confirm it works.
2. Replace the demo's image loop with the `frames()` function above. Process frames in batches.
3. From each result, keep the hand side, box, 2D points, and 3D joints or mesh.
4. Save them in the same `hands.jsonl` format, so steps 4–7 work unchanged.

Step 4: Keep IDs and smooth the points

Match hands between frames by side and box overlap, or use ByteTrack. Then smooth each point to remove jitter. A simple moving average is a fine start; the One-Euro filter is better because it smooths more when the hand is still and less when it moves fast:

```
import math
class OneEuro:
    def __init__(self, min_cutoff=1.0, beta=0.01):
        self.mc, self.beta, self.x, self.dx, self.t = min_cutoff, beta, None,
0.0, None
    @staticmethod
    def _alpha(cutoff, dt):
        r = 2 * math.pi * cutoff * dt
        return r / (r + 1)
    def __call__(self, x, t):
        if self.x is None:
            self.x, self.t = x, t
            return x
        dt = max(t - self.t, 1e-6); self.t = t
        self.dx += self._alpha(1.0, dt) * ((x - self.x) / dt - self.dx)
        self.x += self._alpha(self.mc + self.beta * abs(self.dx), dt) * (x -
self.x)
        return self.x
```

Use one filter per coordinate of each point. If you skipped frames, fill them in by averaging the frames on either side.

Step 5: Detect contact with objects (PS2)

A simple rule to start from. Try to beat it.

```
TIPS = [4, 8, 12, 16, 20]    # fingertip point numbers in MediaPipe
def touching(hand, objects, margin=0.15):
    x1, y1, x2, y2 = hand["bbox"]
    pad = margin * max(x2 - x1, y2 - y1)
    for obj in objects:                    # boxes from YOLO or Grounding
DINO
        ox1, oy1, ox2, oy2 = obj["bbox"]
        near = sum(ox1 - pad < hand["kp2d"][i][0] < ox2 + pad and
                   oy1 - pad < hand["kp2d"][i][1] < oy2 + pad for i in TIPS)
        if near >= 2:
            return obj
    return None
```

A contact that lasts only one or two frames is probably noise. Require it to hold for several frames before you record a grab.

Step 6: Build a task-step timeline with an API (PS3)

Get a free API key from Google AI Studio and set it as the environment variable `GEMINI_API_KEY`.

```
import time, json
from google import genai
from google.genai import types

client = genai.Client()
f = client.files.upload(file="clips_480p/factory_001.mp4")
while f.state.name == "PROCESSING":
    time.sleep(5)
    f = client.files.get(name=f.name)

prompt = """This is first-person video from a head-mounted camera in a
factory.
List every hand action in order as a JSON array. Each item has:
start_s, end_s (seconds), verb (e.g. pick, place, fasten, inspect, press),
object (what is handled), hand (left, right or both)."""

resp = client.models.generate_content(
    model="gemini-3.5-flash",     # use a current model name from the Gemini
docs
    contents=[f, prompt],

config=types.GenerateContentConfig(response_mime_type="application/json"))
json.dump(json.loads(resp.text), open("actions_factory_001.json", "w"),
indent=2)
client.files.delete(name=f.name)   # data rule: remove the upload when done
```

To improve it, ask about short windows around each grab from step 5 instead of the whole clip. You can also give the model a fixed list of verbs, so every clip uses the same words.

Step 7: Draw results and record your demo

```
import json, cv2
from mediapipe.tasks.python.vision import HandLandmarksConnections as C

rows = {r["frame"]: r for r in map(json.loads, open("hands.jsonl"))}
cap = cv2.VideoCapture("clips_480p/factory_001.mp4")
fps = cap.get(cv2.CAP_PROP_FPS)
w, h = int(cap.get(3)), int(cap.get(4))
out = cv2.VideoWriter("demo.mp4", cv2.VideoWriter_fourcc(*"mp4v"), fps, (w,
h))
i, last = 0, None
```

```
while True:
    ok, img = cap.read()
    if not ok:
        break
    last = rows.get(i, last)            # reuse the last result on skipped
frames
    for hand in (last or {}).get("hands", []):
        kp = [(int(x), int(y)) for x, y in hand["kp2d"]]
        colour = (0, 200, 0) if hand["side"] == "left" else (0, 120, 255)
        for c in C.HAND_CONNECTIONS:
            cv2.line(img, kp[c.start], kp[c.end], colour, 2)
        cv2.putText(img, hand["side"], kp[0], cv2.FONT_HERSHEY_SIMPLEX, 0.6,
colour, 2)
    out.write(img)
    i += 1
out.release()
```

If `HandLandmarksConnections` is missing in your MediaPipe version, connect the points with your own list of 21 joint pairs instead.

Suggested plan for the 24 hours

Hours	Goal
0–3	Setup, minimum goal working on one clip, first demo video
3–10	Main improvement for your problem statement
10–16	Run on all clips; collect failure cases and fix the biggest one
16–21	Speed and polish; demo clips processed; numbers for your slides
21–23	README, slides, demo video; push code before the freeze
23–24	Rehearse the presentation

Submission and judging

A panel of judges scores every team. There is no automatic leaderboard. Judges score what they see in your demo, your code and your answers to their questions.

Submit before the code freeze:

What	Where	Details
Code	Public GitHub repo	A README with setup steps, the one command that runs your pipeline, your problem statement(s), and whether you used a laptop or a cloud GPU. No videos, frames or screenshots from the data in the repo.
Demo videos	Google Drive folder shared during the event	Your processed demo clips with results drawn on them, 2 minutes or less in total. Upload them only to this folder, never publicly.
Presentation	Same Google Drive location shared during the event	5–6 slides: problem, approach, results, what failed, what you would do next. Put your GitHub repo link on the first slide.

Final presentation. 5 minutes of presentation and live demo, then 3 minutes of questions from the judges. Every team member should be ready to explain any part of the code.

Judging rubric

Criterion	Points	What judges look for
Hand-tracking accuracy	30	Every hand found, points on the right joints, left and right correct, stable through blur, gloves and occlusion on the demo clips
Results for your problem statement	20	Contacts, task steps, 3D motion or your tool working correctly on real factory footage
Technical approach	15	Sensible model choices, clearly explained; real work beyond the starter code
Usefulness to industry	10	Would a factory or robotics team actually use this? Is the output clean and easy to use?
Innovation	10	A new idea, a clever combination of tools, or a smart fix for a hard case
Efficiency	5	Speed and cost for the hardware used
Presentation and honesty	10	A clear demo, failure cases shown honestly, and good answers to questions

Criterion	Points	What judges look for
Total	**100**	

Numbers help your case: the share of frames where hands were found, accuracy on frames you labelled yourself, and frames per second. Ties are broken by the hand-tracking accuracy score.

Rules

- Teams of 2–4 students. A student can be in only one team.
- All project code must be written during the 24 hours. You may prepare beforehand by installing tools, downloading model weights and reading papers.
- Any open-source library, pretrained model or AI API is allowed. List them all in your README.
- AI coding assistants are allowed, but judges will ask you to explain your code.
- Follow the data-use rules in The data section. Breaking them means disqualification.
- The judges' decision is final.

FAQ

- **I have never done computer vision. Can I take part?** Yes. Pick PS1 or PS3. The starter code gets you to the minimum goal, and a clear, honest beginner project can score well.
- **I don't have a GPU.** You don't need one. Use the laptop path, or a free Colab or Kaggle GPU for heavy models.
- **Do I need to pay for an API?** No. The Gemini API has a free tier through Google AI Studio. Free tiers have rate limits, so start early and save every response.
- **How do I know if my results are correct, without labels?** Watch your output videos. For extra credit, label a few minutes of one clip yourself and measure how often you are right.
- **MediaPipe finds no hands in some frames. What do I do?** Save an empty list for that frame and keep going. Then fix those gaps: missed hands count directly against your accuracy score.
- **Can we change problem statements during the event?** Yes, but tell the organisers.

Glossary

Term	Meaning
Egocentric video	Video from a camera worn by the person, showing their point of view
Keypoints / landmarks	Specific points on the hand, such as fingertips and knuckles. MediaPipe gives 21 per hand.
Bounding box (bbox)	The smallest rectangle around a hand or object: `[x1, y1, x2, y2]`
Tracking	Keeping the same ID on a hand or object from frame to frame
Jitter	Points shaking slightly between frames even when the hand is still
Hand mesh / MANO	A full 3D surface model of the hand, not just points
Open-vocabulary detector	A detector that finds any object you name in text, without extra training
Vision-language model (VLM)	An AI model that takes images or video and answers in text
FPS	Frames per second, either of the video or of your processing speed
ONNX / OpenVINO	Tools that make models run faster on a normal CPU

Resources

- MediaPipe Hand Landmarker guide for Python
- Gemini API: video understanding
- WiLoR and HaMeR: 3D hand reconstruction
- HaWoR: world-space hand motion from head-camera video
- SAM 2: object segmentation and tracking in video
- EgoHandICL (ICLR 2026): a summary of where today's hand models still fail
- Ego4D and EPIC-KITCHENS: public egocentric research, for background reading